SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Emergency Medicine Delivery Using Greedy Best-First Search and A* Search

Problem Statement

A government deploys AI-powered drones to deliver emergency medicines to flood-affected regions. The environment is uncertain because roads may become blocked, weather conditions may change, and movement costs vary depending on terrain and weather. The drone uses straight-line distance as a heuristic to estimate its distance from the destination. The primary objective is to find a route that minimizes delivery time and risk while responding quickly to changing conditions.

Greedy Best-First Search and A* Search are two informed search algorithms that use heuristic information to guide the search towards the goal. However, they differ in how they select the next node and how they consider the cost of the path already travelled.

i. Behaviour of Greedy Best-First Search and Its Strengths and Weaknesses

Greedy Best-First Search selects the node that appears closest to the goal according to the heuristic function. In this scenario, the drone uses straight-line distance to estimate how far each possible position is from the medicine delivery location.

The heuristic function is:

f(n) = h(n)

Where:

- h(n) = estimated straight-line distance from the current node to the destination.
- f(n) = evaluation value used to select the next node.

Working Procedure

Step 1: The drone identifies its current position and the destination.

Step 2: It calculates the straight-line distance from the current position to each available neighbouring position.

Step 3: It selects the node with the smallest heuristic value.

Step 4: It continues moving towards the selected node while avoiding known blocked paths.

Step 5: If a blocked path is discovered, the drone updates its available routes and performs another search.

Example

Suppose the drone has two possible routes:

- Route A appears to be 2 km from the destination according to the straight-line estimate, but it passes through a flooded area.
- Route B appears to be 3 km from the destination, but it follows a safer route.

Greedy Best-First Search may select Route A because its heuristic value is smaller. However, if the route is blocked or dangerous, the drone may need to reconsider its decision.

Strengths

1. **Fast search:** It prioritizes nodes that appear closer to the goal and can reach a promising route quickly.
2. **Goal-directed behaviour:** It uses the heuristic to avoid exploring nodes that appear far from the destination.
3. **Useful under time constraints:** It may provide a quick route when an approximate solution is acceptable.
4. **Simple implementation:** The algorithm can be implemented using a priority queue ordered by heuristic value.

Weaknesses

1. **Ignores the cost already travelled:** It may select a route that looks short but has high travel time or risk.
2. **Does not guarantee an optimal solution:** The route with the smallest estimated distance may not have the lowest total cost.
3. **Affected by inaccurate information:** Straight-line distance does not account for blocked roads, strong winds, flooded regions, or difficult terrain.
4. **Requires replanning:** When the drone discovers a blocked path, it must update its search and select another route.

Therefore, Greedy Best-First Search can quickly identify promising routes, but it may not be the safest or least-cost choice in a flood-affected environment.

ii. Working of A* Search and Its Cost-Heuristic Balance

A* Search combines the actual cost of reaching a node with the estimated cost of reaching the goal. This allows the drone to consider both the journey already completed and the distance remaining.

The evaluation function is:

f(n) = g(n) + h(n)

Where:

- g(n) = actual cumulative cost from the starting position to node n.
- h(n) = estimated cost from node n to the destination.
- f(n) = estimated total cost of a solution passing through node n.

In the drone scenario, g(n) may include travel time, energy consumption, and assigned risk costs. The heuristic h(n) estimates the remaining travel cost using straight-line distance, provided the estimate is expressed in compatible cost units.

Working Procedure

Step 1: The drone initializes the starting node with g(n) = 0.

Step 2: It calculates the heuristic distance from the starting node to the destination.

Step 3: It maintains a priority queue of available nodes, ordered by the smallest f(n) value.

Step 4: It selects the node with the smallest estimated total cost and examines its neighbouring positions.

Step 5: For each valid neighbour, it calculates the new cumulative cost and adds the heuristic estimate.

Step 6: It updates the node's cost if a cheaper route to that node is discovered.

Step 7: When the destination is reached, it reconstructs the route using the recorded parent nodes.

Step 8: If new information reveals a blocked path or changed travel cost, the drone updates the route information and replans.

Example

Consider two possible routes:

Factor	Route A	Route B

Actual cost so far, g(n)	8	4
Estimated remaining cost, h(n)	2	5
Total estimated cost, f(n)	10	9

For Route A:

f(n) = 8 + 2 = 10

For Route B:

f(n) = 4 + 5 = 9

A* selects Route B because its estimated total cost is lower, even though Route A appears closer to the destination.

This demonstrates how A* balances the cost already incurred with the estimated remaining cost.

Advantages of A* Search

1. **Cost-sensitive decisions:** It considers the cumulative cost of travel rather than only the remaining distance.
2. **Optimality under suitable conditions:** A* finds an optimal path when the heuristic is admissible and the algorithm is implemented appropriately. With a consistent heuristic, graph-search implementations can efficiently avoid unnecessary reopening of nodes.
3. **Flexible cost representation:** Travel time, energy usage, and risk can be incorporated into the path cost.
4. **Improved route selection:** It can avoid routes that appear short but have high cumulative costs.

However, A* may require considerable memory and computation when the search space is large. Dynamic changes also require cost updates and replanning.

iii. Impact of Heuristic Accuracy on Both Algorithms

A heuristic function estimates the remaining cost from a node to the goal. Its accuracy directly influences how effectively a search algorithm identifies promising routes.

Impact on Greedy Best-First Search

Greedy Best-First Search depends entirely on h(n) to select the next node. Therefore, an inaccurate heuristic can cause the drone to follow an unsuitable route.

For example, a route may have a small straight-line distance but contain a flooded road or a region affected by strong winds. If these conditions are not represented in the heuristic, the drone may incorrectly consider that route preferable.

A more accurate heuristic that accounts for known environmental conditions can improve route selection. However, even a good heuristic cannot guarantee optimality because Greedy Best-First Search ignores the actual cost already incurred.

Impact on A* Search

A* uses both g(n) and h(n). Consequently, errors in the heuristic do not affect the algorithm in exactly the same way as they affect Greedy Best-First Search.

An accurate and admissible heuristic can guide A* towards the goal while preserving optimality under the appropriate conditions.

An admissible heuristic never overestimates the true minimum remaining cost. For example, straight-line distance can serve as a lower-bound estimate of travel distance when movement is restricted to routes that cannot be shorter than the straight-line distance.

However, if the heuristic is expressed as distance while g(n) represents time or a combined time-risk cost, the two quantities are not directly comparable. The heuristic must be converted into compatible cost units.

A heuristic that overestimates the remaining cost may cause standard A* to lose its optimality guarantee.

Comparison

Factor	Greedy Best-First Search	A* Search
Evaluation function	f(n) = h(n)	f(n) = g(n) + h(n)
Dependence on heuristic	Very high	Combined with actual path cost
Optimality	Not guaranteed	Guaranteed under suitable conditions
Effect of poor heuristic	May follow a misleading route	May become less efficient or lose optimality if admissibility is violated

Route-cost consideration	Ignores cumulative path cost	Considers cumulative path cost

Thus, heuristic accuracy is important for both algorithms, but A* provides a stronger basis for cost-sensitive route selection.

iv. Effect of Dynamic Changes and Blocked Paths

The drone operates in a dynamic environment where roads may suddenly become inaccessible and weather conditions can change travel costs.

These changes affect both the available routes and the estimated cost of reaching the destination.

Challenges

1. **Invalidated routes:** A path that was previously available may become blocked.
2. **Changed movement costs:** Strong winds, flooding, or poor visibility may increase travel time and energy consumption.
3. **Repeated computation:** The algorithm may need to search again whenever significant changes occur.
4. **Incomplete information:** The drone may not immediately receive information about every blocked route.
5. **Safety risks:** Continuing along an outdated route may expose the drone to dangerous conditions.

Handling Dynamic Changes

When the drone detects a blocked path, it should mark the affected route as unavailable and update its internal map.

It must then reconsider the remaining available routes. If a previously calculated path is no longer feasible, the drone should perform a new search or use an incremental replanning method.

When weather changes the movement cost, the drone should update the corresponding edge costs and recalculate the best route.

A* can be combined with incremental search methods, such as D* Lite, to reuse previous search information when planning in changing environments.

Greedy Best-First Search can also replan after a blockage, but it may repeatedly choose routes that appear close to the goal without adequately accounting for their total travel costs.

For safety, the drone should prioritize current obstacle information, maintain sufficient energy reserves, and avoid routes that violate safety constraints.

v. Recommended Algorithm and Justification

*Recommended Approach: A Search with Dynamic Replanning**

A* Search is the more suitable primary algorithm because the drone must consider travel time, terrain costs, and risk rather than simply minimizing geometric distance.

Justification

1. **Cost-sensitive route planning:** A* evaluates the cumulative cost and estimated remaining cost, allowing it to compare alternative routes more effectively.
2. **Optimality:** With an admissible heuristic and a suitable implementation, A* can find a minimum-cost path in the represented environment.
3. **Safety considerations:** Risk can be included in the path cost, while dangerous routes can be excluded through hard safety constraints.
4. **Dynamic adaptability:** The drone can update its map and replan when paths become blocked or movement costs change.
5. **Heuristic guidance:** Straight-line distance can reduce unnecessary exploration when it provides a valid lower-bound estimate.

For strict real-time requirements, the system can use bounded search, incremental replanning, and a reliable emergency fallback. D* Lite may be considered when repeated route changes make incremental replanning more efficient.

A* does not automatically guarantee safety or instant decisions in every environment. Its effectiveness depends on accurate cost information, suitable heuristic design, sufficient computational resources, and timely replanning.

Result

Greedy Best-First Search can identify promising routes quickly, but it may ignore important travel costs and select a route that is unsafe or inefficient. A* Search provides a more balanced approach by combining cumulative path cost with an estimate of the remaining cost.

Therefore, A* Search with dynamic replanning is recommended for emergency medicine delivery, provided that the drone updates environmental information regularly and enforces safety constraints. This approach balances route quality, computational efficiency, and adaptability to changing conditions.

2. Smart City Traffic Signal Optimization Using Hill Climbing, Simulated Annealing, and Genetic Algorithms

Problem Statement

A smart city uses an AI-based traffic management system to optimize signal timings across hundreds of intersections. The system aims to minimize total vehicle waiting time, fuel consumption, and traffic congestion.

The environment changes continuously as vehicle arrivals, traffic density, and road conditions vary. The number of possible signal-timing combinations is extremely large, making exhaustive search inefficient. Furthermore, a search algorithm may become trapped in a locally optimal solution that is not the best solution for the entire traffic network.

Optimization algorithms such as Hill Climbing, Simulated Annealing, and Genetic Algorithms can be used to search for effective signal-timing configurations.

Formulation as an Optimization Problem

The traffic signal system can be represented as an optimization problem in which a candidate solution specifies the signal timings at all controlled intersections.

Decision Variables

The decision variables may include:

- Green-light duration for each traffic direction.
- Red-light duration for each traffic direction.
- Signal phase order.
- Coordination offsets between neighbouring intersections.
- Timing adjustments made during each update cycle.

Let X represent a complete configuration of signal timings across the network.

Objective Function

The system aims to minimize a weighted combination of waiting time, fuel consumption, and congestion.

Minimize:

$J(X) = \alpha W(X) + \beta F(X) + \gamma C(X)$

Where:

- $J(X)$ = total objective cost.
- $W(X)$ = total vehicle waiting time.
- $F(X)$ = estimated fuel consumption.
- $C(X)$ = traffic congestion measure.
- α, β, and γ = non-negative weights representing the importance of each objective.

The weights must be selected carefully so that the measures are comparable and reflect the city's priorities.

Constraints

The solution must satisfy the following constraints:

1. Green-light durations must remain within permitted minimum and maximum limits.
2. Conflicting traffic movements must not receive green signals simultaneously.
3. Pedestrian crossing requirements must be satisfied.
4. Emergency vehicle priority must be supported where required.
5. Signal transitions must follow safe operating rules.
6. The timing plan must be feasible for every controlled intersection.

A candidate solution is considered better when it reduces the objective function while satisfying the safety and operational constraints.

Why Traditional Search Is Inefficient

Traditional exhaustive search attempts to examine all possible combinations of signal timings. If each of hundreds of intersections has several possible timing configurations, the total number of combinations grows exponentially.

For example, if each of 100 intersections has 10 candidate configurations, the total number of network configurations is:

10^{100}

Evaluating every possibility is computationally impractical.

Other challenges include:

1. **Large search space:** The number of possible timing configurations increases rapidly with the number of intersections.

2. **Dynamic traffic conditions:** A configuration that works well during one period may perform poorly when traffic changes.
3. **Expensive evaluation:** Estimating network-wide waiting time and fuel consumption may require simulation.
4. **Local optima:** A solution that improves one intersection may worsen traffic at neighbouring intersections.
5. **Real-time requirements:** The system must produce useful decisions within a limited time.

Therefore, heuristic and metaheuristic optimization methods are more practical than exhaustive search for large traffic networks.

i. Working of Hill Climbing in Traffic Optimization

Hill Climbing is a local search algorithm that repeatedly improves the current solution by examining neighbouring solutions.

For this problem, the current solution represents the signal-timing configuration across the city. A neighbouring solution can be created by changing one or more green-light durations or coordination offsets.

Working Procedure

Step 1: Generate an initial feasible signal-timing configuration.

Step 2: Evaluate the configuration using the objective function J(X).

Step 3: Generate neighbouring configurations by making small changes to selected signal timings.

Step 4: Evaluate the neighbouring configurations.

Step 5: Select a neighbour that reduces the objective cost.

Step 6: Replace the current solution with the improved solution.

Step 7: Repeat the process until no improving neighbour is found or the computational time limit is reached.

Example

Suppose a signal-timing configuration produces the following estimated objective cost:

Current configuration: J(X) = 150

After adjusting a green-light duration:

New configuration: J(X') = 130

Since the objective cost decreases from 150 to 130, the new configuration is selected.

The process continues until no neighbouring configuration provides a further improvement.

Advantages

1. **Simple implementation:** The algorithm is easy to understand and implement.
2. **Low memory requirement:** It stores the current solution and a limited number of neighbouring solutions.
3. **Fast local improvement:** It can quickly improve an initial timing plan.
4. **Useful for frequent updates:** Small changes can be evaluated and applied without searching the entire solution space.

Limitations

1. It may stop at a local optimum.
2. It may struggle when several changes must be made together to improve network performance.
3. Its result depends on the initial configuration.
4. It may not adapt well if traffic conditions change faster than the optimization process.
5. It does not guarantee the globally optimal traffic-timing configuration.

Hill Climbing is therefore useful for local adjustments but may be insufficient as the only optimization method for a large city-wide network.

ii. Local Maxima, Plateaus, and Ridges

Hill Climbing can encounter different difficulties while searching the traffic-timing solution space.

Since this problem is formulated as minimization, the following concepts can be interpreted using a cost function. A local minimum is a solution that is better than its immediate neighbours but may not be globally optimal. Equivalently, if the negative cost is treated as a fitness function to maximize, the corresponding difficulty is a local maximum.

1. Local Maxima or Local Minima

A local optimum occurs when none of the immediate neighbouring solutions improves the current solution, even though a better solution exists elsewhere.

Real-world interpretation:

Suppose the system adjusts one intersection to reduce waiting time. This change improves that intersection but increases congestion at a nearby junction. Hill Climbing may settle on a configuration that cannot be improved by a small individual adjustment, even though coordinated changes across several intersections could reduce total city-wide congestion.

Effect: The algorithm may stop before finding a better overall timing plan.

2. Plateaus

A plateau is a region of the search space where many neighbouring solutions have the same or nearly identical objective values.

Real-world interpretation:

Changing a green-light duration by one second may have almost no measurable effect on traffic waiting time because the change is too small to affect vehicle flow significantly.

The algorithm may repeatedly encounter equally performing configurations and struggle to make progress.

Effect: The search may stop unnecessarily or spend time exploring solutions that provide no meaningful improvement.

3. Ridges

A ridge is a region where improvement requires a coordinated sequence of changes rather than a single local adjustment.

Real-world interpretation:

A major road may require synchronized green signals at several consecutive intersections. Adjusting only one signal may worsen traffic, while changing all the relevant signals together may improve the entire corridor.

Effect: Hill Climbing may fail to discover the coordinated improvement because individual changes do not immediately reduce the objective cost.

These problems show why a more flexible optimization strategy may be required for large and interconnected traffic networks.

iii. How Simulated Annealing and Genetic Algorithms Overcome These Limitations

A. Simulated Annealing

Simulated Annealing is a probabilistic optimization algorithm inspired by the physical process of cooling heated materials.

Unlike basic Hill Climbing, it can sometimes accept a worse solution temporarily. This allows it to escape local optima and explore other regions of the search space.

For minimization, if the current cost is J(X) and a neighbouring solution has cost J(X'), then:

$\Delta J = J(X') - J(X)$

If $\Delta J < 0$, the new solution improves the objective and is accepted.

If $\Delta J > 0$, the new solution may still be accepted with probability:

$P = e^{(-\Delta J/T)}$

Where:

- P = probability of accepting a worse solution.
- ΔJ = increase in objective cost.
- T = temperature parameter controlling the acceptance probability.

At a high temperature, the algorithm is more likely to accept worse solutions and explore widely. As the temperature decreases, it becomes more selective and focuses on improving the current solution.

Working Procedure

Step 1: Generate an initial feasible traffic-timing configuration.

Step 2: Calculate its objective cost.

Step 3: Generate a neighbouring configuration by adjusting selected signal timings.

Step 4: Calculate the change in objective cost.

Step 5: Accept the new configuration if it improves the objective. Otherwise, accept it with the probability determined by the temperature.

Step 6: Gradually reduce the temperature according to a cooling schedule.

Step 7: Continue until the stopping condition or time limit is reached.

Advantages

1. It can escape local optima by accepting some worse solutions.
2. It explores a wider range of candidate configurations than basic Hill Climbing.
3. It requires relatively little memory.

4. It is suitable for optimization problems with large and complex search spaces.

Limitations

1. Its performance depends on the cooling schedule.
2. A slow cooling schedule may require considerable computation.
3. It does not guarantee a globally optimal result within a fixed real-time deadline.
4. Candidate solutions must remain feasible and satisfy traffic-safety constraints.

B. Genetic Algorithms

A Genetic Algorithm is a population-based optimization method inspired by natural selection and genetics.

Instead of maintaining only one solution, it maintains a population of candidate signal-timing configurations. Better-performing candidates have a greater chance of contributing to future generations.

Representation

Each candidate solution, called a chromosome, can encode the green-light durations, phase timings, and coordination offsets for selected intersections.

For example:

Chromosome A = [30, 40, 25, 35, 45]

Each value represents a selected timing parameter. The exact representation depends on the traffic network.

Working Procedure

Step 1: Generate an initial population of feasible signal-timing configurations.

Step 2: Evaluate each configuration using the objective function.

Step 3: Calculate fitness so that configurations with lower objective costs receive better fitness scores.

Step 4: Select suitable parent configurations.

Step 5: Apply crossover to combine parts of two parent configurations.

Step 6: Apply mutation to introduce small random changes in selected timing parameters.

Step 7: Repair or reject any offspring that violate safety and operational constraints.

Step 8: Evaluate the new population and retain strong candidate solutions.

Step 9: Repeat the process until the stopping condition is reached.

Example

Suppose two candidate configurations are:

Parent A = [30, 40, 25, 35]

Parent B = [35, 45, 30, 40]

Crossover may produce:

Offspring = [30, 40, 30, 40]

Mutation may then change one timing value to generate another candidate.

Each candidate is evaluated to determine whether it reduces waiting time, fuel consumption, and congestion.

Advantages

1. **Population-based exploration:** Multiple candidate solutions are evaluated simultaneously.
2. **Reduced dependence on one starting point:** The population can explore different regions of the search space.
3. **Crossover and mutation:** These operations can produce new configurations that are not simple local adjustments of one current solution.
4. **Suitable for complex objectives:** The method can optimize several interacting timing parameters.
5. **Potential for parallel evaluation:** Candidate configurations can be evaluated simultaneously when sufficient computing resources are available.

Limitations

1. Fitness evaluation can be computationally expensive.
2. Population size and genetic parameters affect performance.
3. The algorithm does not guarantee a globally optimal solution.
4. Poorly designed crossover or mutation can produce infeasible timing configurations.
5. It may require careful constraint handling to preserve traffic-safety rules.

Comparison

Factor	Hill Climbing	Simulated Annealing	Genetic Algorithm

Search strategy	Single-solution local search	Single-solution probabilistic search	Population-based search
Escaping local optima	Limited	Can accept worse solutions temporarily	Explores through selection, crossover, and mutation
Memory requirement	Low	Low	Higher due to population storage
Computational cost	Usually low	Depends on cooling schedule	Depends on population and generations
Suitability	Quick local adjustments	Escaping local optima with limited memory	Exploring complex combinations across a large search space

iv. Recommended Approach for Scalability and Adaptability

Recommended Approach: Genetic Algorithm with Feasibility Checks and Local Refinement

A Genetic Algorithm is a suitable choice for exploring complex signal-timing configurations across hundreds of interconnected intersections. It can maintain several candidate solutions and search for coordinated changes that may be difficult to discover using basic Hill Climbing.

Justification

1. **Scalability:** The search can be distributed across candidate configurations and divided into smaller regional optimization problems.
2. **Adaptability:** The population can be updated when traffic patterns change, using recent conditions to evaluate new configurations.
3. **Global exploration:** Maintaining multiple candidates reduces dependence on a single starting solution.
4. **Coordinated timing:** Crossover and mutation can generate combinations of signal timings across several intersections.
5. **Constraint handling:** Feasibility checks and repair mechanisms can ensure that generated solutions follow safety rules.

For real-time operation, the system should not wait for a Genetic Algorithm to finish an unrestricted search. It should impose a fixed computation budget and retain the best feasible solution found so far.

The city can use a hierarchical design in which regional controllers optimize groups of nearby intersections while a higher-level controller coordinates major traffic corridors. Local adjustments can then be applied to handle immediate changes.

Simulated Annealing is a reasonable alternative when memory is limited and the system needs a relatively simple method for escaping local optima. However, for a large network with many interacting timing parameters, a Genetic Algorithm offers a useful population-based search strategy.

Neither method guarantees a globally optimal solution within a fixed time. Their effectiveness depends on objective-function design, candidate evaluation, and the frequency of traffic updates.

Result

Traffic signal optimization can be formulated as a constrained minimization problem involving waiting time, fuel consumption, and congestion. Hill Climbing can provide fast local improvements but may become trapped in local optima, plateaus, and ridges.

Simulated Annealing addresses some of these limitations by occasionally accepting worse solutions, while Genetic Algorithms explore multiple candidate configurations using selection, crossover, and mutation.

Therefore, a Genetic Algorithm with feasibility checks, regional optimization, and local refinement is recommended for large-scale smart city traffic management. A fixed computation budget and frequent evaluation using updated traffic information can help the system balance optimization quality with real-time responsiveness.

3. Autonomous Mars Rover Using Online Search

Problem Statement

An autonomous rover is deployed on Mars to explore unknown terrain and collect scientific samples. It must reach target locations while avoiding craters, rocks, and other hazards. The rover has limited sensing capability and does not possess a complete map of the environment.

As the rover moves, it discovers new terrain information and updates its internal representation. Therefore, it must make decisions using incomplete information and revise its route when new obstacles or hazards are discovered.

Online search is appropriate because the rover must interleave exploration, sensing, action, and planning instead of calculating an entire route from a fully known map before moving.

i. Why an Online Search Agent Is Required Instead of Offline Search

Offline search assumes that the agent has access to a sufficiently complete model of the environment before beginning the search. It can calculate a route and then execute the plan.

However, the Mars rover does not initially know the exact condition of every part of the terrain. Its sensors provide only limited information, and some obstacles may become visible only when the rover approaches them.

An online search agent performs actions while learning about the environment. It repeatedly observes its surroundings, updates its knowledge, selects an action, and continues planning.

Working Procedure

Step 1: The rover identifies its current position and target location.

Step 2: It observes nearby terrain using its sensors.

Step 3: It records known safe areas, obstacles, and uncertain regions in its internal map.

Step 4: It selects a feasible movement using the available information.

Step 5: It moves to the selected location and gathers new observations.

Step 6: It updates its internal map when it discovers new terrain features or hazards.

Step 7: If the previous route is blocked or unsafe, it selects an alternative route.

Step 8: It repeats the process until the target is reached or the mission must be suspended for safety.

Example

Suppose the rover identifies a seemingly clear route towards a sample collection site. After moving forward, it discovers a crater that was not visible from its starting position.

An offline plan based on an incomplete map may no longer be usable. An online agent updates the map, marks the crater as hazardous, and searches for another feasible route.

Advantages

1. **Adaptability:** The rover can respond to newly discovered terrain.
2. **Reduced dependence on a complete map:** It can begin operating with partial information.
3. **Continuous learning:** Its internal model becomes more accurate as it explores.
4. **Improved decision-making:** It can reconsider its route when a hazard is detected.
5. **Support for autonomous operation:** It can make local navigation decisions without requiring a complete route to be specified in advance.

Offline search may still be useful for planning through previously mapped regions. However, online search is more appropriate for the rover's exploration of unknown terrain.

ii. Challenges of Partial Observability and Dynamic Environments

1. Partial Observability

Partial observability means that the rover cannot directly observe the complete state of its environment.

Its sensors may reveal nearby rocks, slopes, and craters, but distant terrain may remain unknown.

Challenges:

- The rover may not know whether a distant route is safe.
- Some obstacles may be hidden by terrain features.
- Sensor measurements may be incomplete or uncertain.
- The rover must distinguish between known safe areas and unexplored regions.

Effect on decision-making:

The rover should not assume that an unknown cell is safe. It must gather additional information before committing to a risky movement.

2. Uncertain Terrain Information

Sensor readings may be affected by limited resolution, lighting conditions, dust, or measurement errors.

Effect on decision-making:

The rover should maintain confidence estimates where appropriate and use conservative movement rules when the terrain classification is uncertain.

3. Dynamic or Newly Discovered Obstacles

The environment may be largely static, but the rover's knowledge changes continuously as it explores. Its internal map must reflect new observations, and previously selected routes may become infeasible.

Effect on decision-making:

The rover must update its model and replan when it discovers an obstacle or identifies a route that does not satisfy safety requirements.

4. Limited Computational Resources

The rover has finite processing power, memory, and energy. Searching every possible route may be too expensive.

Effect on decision-making:

The rover must prioritize useful observations and choose search methods that can produce safe, practical decisions within its available resources.

5. Communication Delays

Communication between Mars and Earth introduces delays, so the rover cannot depend on receiving immediate instructions for every movement.

Effect on decision-making:

The rover must perform local navigation autonomously and handle routine obstacle avoidance without waiting for instructions from Earth.

6. Safety and Mission Constraints

A route that reaches a sample location quickly may expose the rover to steep slopes, unstable terrain, or excessive energy consumption.

Effect on decision-making:

The rover should incorporate hazard avoidance, terrain traversability, energy requirements, and mission priorities into its action-selection process.

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an internal representation of the terrain using information collected through its sensors and movement history.

The model may be represented as a grid map, a topological map, or another spatial representation suitable for navigation.

Step 1: Initialize the Internal Map

The rover begins with its current position, the target location, and any available prior terrain information.

Unknown areas are marked as unexplored rather than automatically classified as safe.

Step 2: Collect Sensor Information

The rover uses cameras, distance sensors, and other available instruments to observe nearby terrain.

It identifies features such as:

- Traversable terrain.
- Rocks and craters.
- Steep slopes.
- Uncertain or unexplored regions.
- Potential sample collection sites.

Step 3: Update the Map

The rover records newly observed terrain information in its internal map.

For example, a previously unknown cell may be classified as blocked after a crater is detected.

The rover should also account for uncertainty in sensor readings instead of treating every observation as perfectly accurate.

Step 4: Assign Traversal Costs

The rover assigns movement costs to available terrain.

A simple cost model may include:

Movement Cost = Distance Cost + Terrain Penalty + Hazard Penalty

Safe, easily traversable terrain receives a lower cost. Steep or difficult terrain receives a higher cost, while prohibited or impassable terrain is excluded.

The exact values depend on the rover's capabilities and mission requirements.

Step 5: Select the Next Action

The rover uses its current map and target information to choose a feasible movement.

It may use an online search strategy that evaluates known routes and identifies promising unexplored regions.

Step 6: Compare Predictions with Observations

After moving, the rover compares the actual terrain observations with its previous expectations.

If it discovers an unexpected obstacle, it corrects the map and updates the affected route costs.

Step 7: Replan

The rover recalculates its route when new information changes the feasibility or cost of its current plan.

It continues this process until the target is reached or the mission must be paused because no safe route is available.

Importance of the Internal Model

The internal model allows the rover to retain information about previously explored areas. Without it, the rover might repeatedly inspect the same locations or make decisions based only on its immediate surroundings.

A continuously updated model improves route planning, supports obstacle avoidance, and reduces unnecessary exploration.

iv. Comparison of Online Search with Uninformed and Informed Search Strategies

A. Online Search

Online search interleaves action and information gathering. The agent does not require a complete map before beginning its exploration.

In the Mars rover scenario, the rover observes nearby terrain, moves to a selected location, updates its map, and replans when necessary.

Advantages:

1. Suitable for unknown environments.
2. Supports decisions based on newly obtained information.
3. Allows route revision when obstacles are discovered.
4. Supports autonomous exploration.

Limitations:

1. It may require additional movements to discover the terrain.
2. Its decisions are limited by the quality of its observations.
3. It cannot guarantee a globally optimal route when significant parts of the environment remain unknown.
4. Poor exploration decisions may increase energy consumption and mission risk.

B. Uninformed Search

Uninformed search methods do not use a heuristic estimate of the distance to the goal. Examples include Breadth-First Search and Uniform Cost Search.

Breadth-First Search explores states level by level and is suitable when all movements have equal costs. Uniform Cost Search expands the node with the lowest cumulative path cost and is appropriate when movement costs differ.

For the rover, uninformed search may be useful in a known map, but it may explore many irrelevant states when the search space is large.

Advantages:

- Does not require a heuristic function.
- UCS can find a minimum-cost path in a known graph with non-negative costs.
- BFS finds the shortest path in terms of the number of moves when all movement costs are equal.

Limitations:

- It may explore many unnecessary states.
- It does not naturally prioritize locations that appear closer to the target.
- Standard implementations assume that the relevant search graph is available.

C. Informed Search

Informed search uses a heuristic to estimate how close a state is to the goal. Examples include Greedy Best-First Search and A* Search.

A* uses:

f(n) = g(n) + h(n)

Where:

- g(n) = actual cost from the starting position.
- h(n) = estimated remaining cost.
- f(n) = estimated total cost of the path through the node.

For the rover, A* can use a suitable distance estimate and terrain-cost model to prioritize promising routes.

Advantages:

- Can reduce unnecessary exploration.
- Uses goal-directed information.
- A* can find an optimal path under suitable heuristic and graph-search conditions.

Limitations:

- The heuristic may be inaccurate in unknown terrain.
- Search costs and available routes may change as the rover explores.
- Standard informed search still requires updates or replanning when the map changes.

Comparison Table

Factor	Online Search	Uninformed Search	Informed Search
Main purpose	Act while learning about the environment	Search without heuristic guidance	Use heuristic information to guide search

Initial map requirement	Can operate with partial information	Standard implementations require a defined search graph	Standard implementations require a defined search graph and heuristic
Heuristic use	Optional, depending on the method	No	Yes
Adaptation to newly discovered terrain	Built into the action-and-observation process	Requires integration with an online mechanism	Requires integration with map updates and replanning
Search efficiency	Depends on exploration strategy	May explore many states	Can reduce exploration with a useful heuristic
Rover suitability	Essential for unknown terrain	Useful for cost calculations in known regions	Useful for goal-directed route planning when a suitable map or model is available

Online search is not mutually exclusive with uninformed or informed search. An online rover can use an informed method to select promising routes while continuously updating its internal model.

v. Recommended Approach and Justification

Recommended Approach: Online Search with Heuristic-Guided Planning and Safety-Based Replanning

The Mars rover should use an online search agent that continuously observes the terrain, updates its internal map, and selects safe movements. A* can be used to guide route planning through the currently known map, while replanning handles newly discovered obstacles.

Justification

1. **Handles partial observability:** The rover can operate without a complete map and gather information as it moves.
2. **Supports adaptability:** It can revise its route when new obstacles or hazardous regions are discovered.
3. **Improves route selection:** Heuristic-guided planning can prioritize routes that appear closer to the target while accounting for cumulative traversal costs.

4. **Supports safety:** Hazardous or impassable terrain can be excluded or assigned suitable penalties.
5. **Reduces unnecessary exploration:** A useful heuristic can guide the rover towards promising areas rather than exploring all known states equally.
6. **Supports autonomous operation:** The rover can make local decisions without depending on continuous communication with Earth.

The rover should maintain a clear distinction between known safe terrain and unexplored terrain. It should not treat unknown areas as guaranteed safe and should stop, gather additional information, or select an alternative route when safety cannot be established.

For repeated replanning in changing maps, incremental methods such as D* Lite may be useful because they can reuse previous search information instead of starting every search from scratch.

The choice of algorithm must also consider limited computing resources, energy reserves, sensing uncertainty, and the consequences of a navigation failure.

Result

Online search is appropriate for the Mars rover because the environment is not completely known in advance. The rover must explore, gather sensor information, update its internal model, and revise its actions when it discovers obstacles or hazards.

An online search agent combined with heuristic-guided planning and safety-based replanning provides an effective approach to autonomous navigation. It supports adaptability and efficient exploration while helping the rover make decisions using the information currently available.

4. University Examination Timetabling Using Constraint Satisfaction Problems

Problem Statement

A university needs to generate an examination timetable for multiple courses, students, faculty members, and examination halls. The timetable must ensure that no student has overlapping examinations, hall capacity is not exceeded, prerequisite subjects are scheduled in the correct order, faculty members are available, and students with special accommodations receive suitable arrangements.

As the number of courses and students increases, the number of possible timetables grows significantly. Constraint Satisfaction Problems provide a systematic way to represent these requirements and search for a valid timetable.

i. Formulation of the Problem as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of three main components:

CSP = (Variables, Domains, Constraints)

- Variables represent the decisions that must be made.
- Domains represent the possible values for each variable.
- Constraints specify which combinations of values are allowed.

For examination timetabling, the university must assign each examination a suitable time slot and examination hall while satisfying all relevant restrictions.

1. Variables

Let each course be represented by a variable.

For example:

X_1 = Examination for Mathematics

X_2 = Examination for Artificial Intelligence

X_3 = Examination for Database Management Systems

More generally, for every course i:

$X_i = (T_i, H_i)$

Where:

- T_i = examination time slot assigned to course i.
- H_i = examination hall assigned to course i.

The timetable must assign a valid time slot and hall to every examination.

2. Domains

The domain of a variable contains all the possible assignments that may be considered for that examination.

For example:

$T_i \in$ {Slot 1, Slot 2, Slot 3, Slot 4}

$H_i \in$ {Hall A, Hall B, Hall C}

The actual domain depends on examination duration, hall availability, faculty schedules, and student accommodation requirements.

If a hall is unavailable during a particular slot, that hall-slot combination must be excluded from the domain.

3. Constraints

Constraints determine whether a proposed timetable is valid.

A. No Student Examination Overlap

If two courses share at least one student, their examinations must not be scheduled at the same time.

For courses i and j:

If StudentSet(i) ∩ StudentSet(j) ≠ ∅, then $T_i \neq T$□.

This prevents a student from being required to attend two examinations simultaneously.

B. Limited Examination Hall Capacity

The number of students assigned to an examination hall must not exceed its capacity.

For every examination i:

Number of Students(i) ≤ Capacity(H_i)

If an examination requires 150 seats, a hall with a capacity of 100 cannot be assigned unless the university provides another valid arrangement.

Where examinations are split across multiple rooms, the model must explicitly represent the allocation of students to those rooms.

C. Subject Ordering Constraints

Some subjects must be scheduled before others.

For example, if Mathematics must be completed before an advanced subject:

T(Mathematics) < T(Advanced Subject)

The ordering relation must be defined according to the university's permitted examination slots and dates.

D. Faculty Availability

An examination must be assigned only to a time when the required faculty or invigilators are available.

If faculty member F is unavailable during Slot 2, examinations requiring that faculty member must not be assigned to Slot 2 when their presence is mandatory.

E. Special Accommodations

Students with special accommodations may require accessible halls, extra examination time, suitable seating, or other approved arrangements.

The timetable must ensure that the assigned slot and facilities satisfy the relevant requirements.

For example, if a student requires an accessible examination hall, the examination must be assigned to a hall that meets the accessibility requirement.

F. Hall and Time Availability

A hall cannot host two incompatible examinations simultaneously. Every examination must be assigned to a hall and slot that are available and suitable for its requirements.

Hard and Soft Constraints

It is useful to distinguish between hard and soft constraints.

Hard constraints must always be satisfied. These include student examination conflicts, hall capacity, mandatory subject ordering, and essential accessibility requirements.

Soft constraints express preferences rather than absolute requirements. Examples include avoiding too many examinations on the same day for a student or distributing examinations evenly across the timetable.

A valid timetable must satisfy all hard constraints. Soft constraints can be optimized to improve the quality of the timetable.

ii. Working of Backtracking Search

Backtracking Search is a systematic CSP algorithm that assigns values to variables one at a time. Whenever an assignment violates a constraint, the algorithm rejects it and tries another value.

Step 1: Initialize the Variables

Create variables for all examinations and identify the available time slots and halls.

Initially, no examination is assigned.

Step 2: Select an Unassigned Variable

Select an examination that has not yet been scheduled.

A useful strategy is the Minimum Remaining Values (MRV) heuristic, which selects the variable with the fewest remaining legal assignments.

This helps identify examinations that are difficult to schedule before their available options become restricted.

Step 3: Select a Domain Value

Choose a feasible combination of time slot and hall from the selected examination's domain.

The value can be chosen using a suitable ordering strategy, such as Least Constraining Value (LCV), which prefers assignments that leave more options available for other examinations.

Step 4: Check the Constraints

Before accepting the assignment, verify that it does not violate any hard constraints.

The checks include:

- No overlapping examinations for shared students.
- Hall capacity and availability.
- Subject ordering requirements.
- Faculty availability.
- Special accommodation requirements.

Step 5: Continue the Search

If the assignment is valid, temporarily assign the examination and select another unassigned variable.

The process continues until all examinations are assigned.

Step 6: Backtrack When Necessary

If no legal value remains for the current variable, the algorithm returns to the previous variable and changes its assignment.

It then continues searching for a valid combination.

Step 7: Complete the Timetable

When all variables are assigned and all constraints are satisfied, the algorithm returns the completed timetable.

If every possible assignment has been explored without success, the algorithm reports that no solution exists under the current domains and constraints.

Illustrative Example

Suppose three examinations have the following relationships:

- Mathematics and Physics share students.
- Physics and Chemistry share students.
- Mathematics and Chemistry do not share students.

Available slots are Slot 1 and Slot 2.

Try the following assignments:

Mathematics = Slot 1

Physics = Slot 1

This assignment is invalid because Mathematics and Physics share students.

Therefore, the algorithm rejects the Physics assignment and tries:

Physics = Slot 2

Now, Chemistry cannot be assigned to Slot 2 because it shares students with Physics. It can be assigned to Slot 1 if its other constraints are satisfied.

The resulting assignment is:

Examination	Assigned Slot
Mathematics	Slot 1
Physics	Slot 2
Chemistry	Slot 1

This example demonstrates how backtracking rejects invalid assignments and continues searching for a valid timetable.

iii. How Constraint Propagation and Forward Checking Improve Efficiency

Constraint propagation reduces the search space by removing values that cannot participate in a valid solution.

Instead of waiting until an entire timetable has been assigned, the system identifies the consequences of each new assignment as early as possible.

Forward Checking

Forward checking is a constraint-propagation technique used during backtracking.

Whenever a variable is assigned a value, the algorithm removes incompatible values from the domains of related unassigned variables.

Working Procedure

Step 1: Select an examination and assign it a time slot.

Step 2: Identify other examinations that share students with the assigned examination.

Step 3: Remove the selected time slot from the domains of those conflicting examinations.

Step 4: Check whether the remaining domains contain at least one valid assignment.

Step 5: If any unassigned examination has an empty domain, reject the current assignment and backtrack immediately.

Step 6: Continue until a complete timetable is found or all assignments have been explored.

Example

Suppose:

Mathematics ∈ {Slot 1, Slot 2, Slot 3}

Physics ∈ {Slot 1, Slot 2, Slot 3}

The two examinations share students.

If Mathematics is assigned Slot 1, forward checking removes Slot 1 from the domain of Physics.

The updated domain becomes:

Physics ∈ {Slot 2, Slot 3}

The algorithm does not need to wait until Physics is selected to discover that Slot 1 is invalid.

Advantages of Forward Checking

1. **Early detection of failure:** An empty domain immediately indicates that the current partial assignment cannot be completed.
2. **Reduced search space:** Invalid values are removed before they are explored.
3. **Fewer backtracking steps:** The algorithm detects many conflicts earlier.
4. **Improved efficiency:** The search avoids exploring combinations that are already known to be invalid.
5. **Better use of constraints:** Student conflicts, hall availability, and faculty restrictions can be used to narrow domains.

Additional Constraint-Propagation Strategies

For larger problems, the university can use stronger propagation techniques, such as maintaining arc consistency, to remove values that have no compatible assignment in related variables.

These methods can reduce unnecessary search, although the propagation process itself requires computational resources.

iv. Real-World Challenges and the Best Strategy for Scalability

Real-World Challenges

1. Large Number of Variables

A university may have hundreds of examinations and many possible time-slot and hall combinations. The search space grows rapidly with the number of variables.

2. Complex Student Conflicts

Students may register for different combinations of elective and core subjects. The system must accurately identify every pair of examinations that share students.

3. Limited Resources

The number of halls, accessible facilities, and available invigilators is limited. Some examinations may require special arrangements.

4. Changing Requirements

Faculty availability, course registrations, examination dates, and hall availability may change after the initial timetable is generated.

5. Conflicting Preferences

A timetable may satisfy all hard constraints but still distribute examinations poorly, creating inconvenient gaps or too many examinations on the same day.

6. Computational Complexity

CSP timetabling is computationally difficult in general. Basic backtracking may become inefficient when the number of examinations and constraints increases.

Recommended Strategy: Backtracking with Constraint Propagation and Heuristics

The most suitable approach is a CSP solver that combines backtracking, forward checking, variable-ordering heuristics, and value-ordering heuristics.

1. Minimum Remaining Values

Select the examination with the fewest legal assignments.

This prioritizes highly constrained examinations and reduces the chance of discovering conflicts late in the search.

2. Degree Heuristic

When several variables have equally small domains, select the examination that constrains the largest number of other unassigned examinations.

This helps address the most influential scheduling decisions first.

3. Least Constraining Value

Prefer the assignment that eliminates the fewest options for other examinations.

This preserves flexibility and increases the chance of completing the timetable.

4. Forward Checking

Remove incompatible time slots and hall assignments from neighbouring domains immediately after each assignment.

This helps detect conflicts early.

5. Separate Hard and Soft Constraints

First ensure that the timetable satisfies all hard constraints. Then improve the solution by minimizing soft-constraint penalties, such as excessive examination clustering or inconvenient gaps.

6. Incremental Rescheduling

If a faculty member becomes unavailable or a hall is closed, the system should attempt to repair the affected part of the timetable rather than rebuilding the entire schedule unnecessarily.

7. Scalability

For very large institutions, the problem can be divided into manageable groups where dependencies permit, while retaining checks for conflicts between groups. A specialized constraint-programming or integer-optimization solver may also be used for large-scale scheduling.

The system must verify the final timetable independently to ensure that every hard constraint is satisfied.

Result

Examination timetabling can be represented as a Constraint Satisfaction Problem with examination variables, domains of available time slots and halls, and constraints involving student conflicts, capacity, subject order, faculty availability, and special accommodations.

Backtracking Search systematically explores assignments, while forward checking removes incompatible domain values and detects failure early.

Therefore, backtracking combined with constraint propagation, Minimum Remaining Values, Degree Heuristic, and Least Constraining Value is an appropriate strategy for generating valid timetables. For large universities, incremental rescheduling and specialized CSP solvers can further improve scalability and adaptability.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

Problem Statement

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, evaluate complex game states, and select effective moves within strict time limits.

The game contains a large number of possible actions at each decision point, creating an enormous game tree. Evaluating every possible sequence of moves is computationally expensive.

Minimax provides a framework for choosing actions by considering the AI's objectives and the opponent's responses. Alpha-Beta Pruning improves its efficiency by eliminating branches that cannot affect the final decision.

i. How the Minimax Algorithm Models the Problem

Minimax is an adversarial search algorithm used in two-player, turn-based, zero-sum games with suitable assumptions about the players' objectives.

It assumes that both players choose actions rationally according to their respective goals. The AI attempts to maximize its evaluation score, while the opponent attempts to minimize it.

In a strategy game, the AI must consider not only the immediate benefit of an action but also the possible responses of the human player.

Game Tree Representation

A game tree represents the possible sequences of moves.

- **Root node:** The current game state.
- **MAX nodes:** States where the AI chooses a move to maximize the evaluation score.
- **MIN nodes:** States where the opponent chooses a move to minimize the AI's evaluation score.
- **Leaf nodes:** Terminal states or states at which the search is stopped and an evaluation function is applied.

Minimax Rule

At a MAX node:

V(n) = max V(child)

At a MIN node:

V(n) = min V(child)

Where V(n) is the value of a game state and V(child) represents the value of a possible successor state.

Working Procedure

Step 1: Represent the current game state as the root of the game tree.

Step 2: Generate the legal actions available to the AI.

Step 3: Simulate each possible action to generate successor states.

Step 4: Generate the opponent's possible responses to each successor state.

Step 5: Continue expanding the tree until a terminal state or a predetermined search depth is reached.

Step 6: Evaluate the leaf states using a utility function or heuristic evaluation function.

Step 7: Propagate the scores upwards through the tree.

At MAX nodes, select the highest child value.

At MIN nodes, select the lowest child value.

Step 8: Choose the action at the root that produces the highest minimax value.

Example

Suppose the AI has two possible actions, A and B. The opponent has two possible responses to each action.

The leaf evaluation scores are:

AI Action	Opponent Response 1	Opponent Response 2	Minimax Value
A	5	2	min(5, 2) = 2
B	3	4	min(3, 4) = 3

The opponent is assumed to select the response that produces the lower score for the AI.

For Action A:

Minimax(A) = min(5, 2) = 2

For Action B:

Minimax(B) = min(3, 4) = 3

The AI selects Action B because:

max(2, 3) = 3

Therefore, Action B is preferred because it provides a better outcome even when the opponent responds optimally.

This demonstrates how Minimax accounts for the opponent's possible decisions rather than selecting a move based only on its immediate reward.

ii. Computational Challenges of Large Game Trees

The main challenge of Minimax is the rapid growth of the game tree.

Let:

- b = average branching factor, or the number of legal actions per state.
- d = search depth, or the number of levels explored.

The approximate number of nodes in a full game tree is:

$$N = 1 + b + b^2 + \ldots + b^d$$

For b > 1:

$$N = (b^{d+1} - 1)/(b - 1)$$

For large trees, the number of states grows exponentially with search depth.

Example

Suppose the AI has an average of 10 legal actions at each level and searches to a depth of 5.

The approximate number of nodes is:

N = 1 + 10 + 100 + 1,000 + 10,000 + 100,000

N = 111,111 nodes

If the search depth increases further, the number of nodes grows rapidly.

Major Challenges

1. Large Branching Factor

Strategy games may provide numerous actions involving unit movement, resource allocation, construction, and attacks.

The large number of possible actions increases the number of states that must be examined.

2. Increasing Search Depth

Deeper search allows the AI to anticipate more future moves, but the number of nodes grows exponentially.

3. Real-Time Constraints

The AI must make decisions within a limited time. A search that takes too long may interrupt gameplay or cause delayed responses.

4. Incomplete Information

Some games contain hidden enemy units, unknown resources, or uncertain future actions. A standard Minimax model does not directly represent every form of uncertainty.

5. Evaluation Accuracy

When the search reaches its depth limit, the AI must estimate the value of non-terminal states. An inaccurate evaluation function can lead to poor decisions even when the search itself is performed correctly.

6. Changing Game States

In a real-time strategy game, units and resources may change while decisions are being calculated. A conventional turn-based Minimax implementation may therefore need adaptation to the game's timing and state-update model.

These challenges make it necessary to reduce unnecessary search and control the depth of exploration.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning is an optimization of Minimax that eliminates branches that cannot influence the final decision.

It produces the same minimax result as standard Minimax when applied correctly to the same search tree. Its advantage is that it can avoid evaluating many unnecessary nodes.

Alpha and Beta Values

Alpha (α): The best value MAX can guarantee from the choices examined so far.

Beta (β): The best value MIN can guarantee from the choices examined so far.

The pruning condition is:

α ≥ β

When this condition is satisfied at a node, the remaining children of that node do not need to be evaluated because they cannot change the final minimax decision.

Working Procedure

Step 1: Initialize α = −∞ and β = +∞ at the root.

Step 2: Explore the game tree using the Minimax search process.

Step 3: At a MAX node, evaluate children and update α to the highest value found so far.

Step 4: At a MIN node, evaluate children and update β to the lowest value found so far.

Step 5: If α ≥ β, stop exploring the remaining children of that node.

Step 6: Continue the search through the remaining relevant branches.

Step 7: Select the best move using the minimax value returned for each root action.

Detailed Example

Consider the following game tree:

- The root is a MAX node.
- It has two children, A and B, which are MIN nodes.
- Each MIN node has two leaf children.

The leaf values are:

A → {3, 5}

B → {2, 9}

Assume the algorithm examines A before B.

Step 1: Evaluate Node A

Since A is a MIN node:

Value(A) = min(3, 5) = 3

The root MAX node updates:

α = max(−∞, 3) = 3

Therefore, the best value found by MAX so far is 3.

Step 2: Begin Evaluating Node B

The first child of B has value 2.

Since B is a MIN node:

β = min(+∞, 2) = 2

At this point:

α = 3

β = 2

Therefore:

α ≥ β

3 ≥ 2

The pruning condition is satisfied.

Step 3: Prune the Remaining Child of B

The second child of B has value 9, but it does not need to be evaluated.

Node B is controlled by MIN. Since MIN already has a response with value 2, the value of B cannot exceed 2 after all its children are considered.

The root MAX node already has an alternative with value 3. Therefore, MAX will not choose B, regardless of the remaining child value.

The branch containing the value 9 can be pruned.

Step 4: Select the Best Action

Value(A) = 3

Value(B) ≤ 2

The root selects A because its minimax value is higher.

Comparison with Standard Minimax

Factor	Minimax	Alpha-Beta Pruning
Search principle	Evaluates the game tree using MAX and MIN decisions	Uses the same minimax decisions while eliminating irrelevant branches
Final decision	Minimax-optimal for the searched tree	Same result as Minimax for the same tree
Nodes evaluated	May evaluate all nodes to the search depth	Can avoid evaluating some nodes
Computational efficiency	Lower when the tree is large	Higher when useful pruning occurs
Dependence on move ordering	Does not affect the final minimax value	Good move ordering can greatly improve pruning efficiency

Computational Complexity

For a branching factor b and depth d, standard Minimax has time complexity approximately:

$O(b^d)$

In the worst case, Alpha-Beta Pruning also has time complexity:

$O(b^d)$

With ideal move ordering, its effective time complexity can approach:

$O(b^{d/2})$

This means that, under favourable conditions, Alpha-Beta Pruning can search much deeper than standard Minimax within a similar computational budget.

However, the improvement depends on the order in which moves are explored. Poor move ordering may result in little pruning.

iv. Design of an Evaluation Function and Justification of Factors

In a large strategy game, searching all the way to a terminal state may be impossible within the available time. Therefore, the AI needs a heuristic evaluation function to estimate the desirability of a non-terminal game state.

A suitable evaluation function should consider both immediate advantages and long-term strategic factors.

Proposed Evaluation Function

$E(s) = 5M + 4R + 3T + 4P + 6O − 5D − 3C$

Where:

- M = military strength advantage.
- R = resource advantage.
- T = territory control advantage.
- P = production capacity advantage.
- O = objective or mission progress advantage.
- D = damage and losses suffered by the AI.
- C = critical resource or infrastructure losses.

The numerical weights are illustrative. They must be adjusted according to the actual rules and strategic priorities of the game.

Each variable represents the AI's advantage relative to the opponent. Positive values indicate an advantage for the AI, while negative values indicate an advantage for the opponent.

Factors Considered

1. Military Strength

Military strength measures the effectiveness and quantity of available combat units.

A larger or better-equipped force can improve the AI's ability to defend territory, attack opponents, and achieve objectives.

This factor receives a significant weight because combat capability may directly determine the outcome of the game.

2. Resource Advantage

Resources may include currency, minerals, energy, or other materials required to construct units and buildings.

A resource advantage can provide greater flexibility in future decisions.

However, resources should not be valued independently of the game situation. Excess resources may have limited value when the game is close to ending.

3. Territory Control

Territory control measures the strategically useful area controlled by the AI.

Controlling important locations may improve resource collection, movement, defensive positions, and access to objectives.

Therefore, the evaluation function should distinguish between valuable territory and areas that provide little strategic benefit.

4. Production Capacity

Production capacity represents the AI's ability to create additional units, buildings, or resources.

A strong production system may provide a long-term advantage even when the AI's current military strength is limited.

This factor helps the AI recognize the strategic value of investment and infrastructure.

5. Objective Progress

Objective progress measures how close the AI is to achieving the game's primary objectives.

Examples include capturing a location, protecting an important unit, completing a mission, or destroying a designated target.

This factor ensures that the AI does not focus only on resources or military strength while neglecting the actual goal of the game.

6. Damage and Unit Losses

Damage and losses reduce the AI's future capabilities.

A strategy that wins a small battle but loses several valuable units may be worse than a strategy that achieves the objective with fewer losses.

This penalty encourages the AI to consider the long-term consequences of its actions.

7. Critical Resource and Infrastructure Losses

Losing a central base, essential production building, or unique resource source may significantly weaken future performance.

This factor allows the AI to distinguish ordinary losses from events that threaten its ability to continue playing effectively.

Importance of Weight Selection

The weights determine the relative importance of each factor. They should reflect the game's actual mechanics.

For example, if capturing an objective immediately wins the game, objective progress should receive greater priority than ordinary resource accumulation.

Similarly, if the game rewards economic development, resource advantage and production capacity may become more important during the early stages.

The evaluation function should also avoid double-counting related factors. For example, military strength and unit losses should be defined consistently so that the same advantage or penalty is not counted twice.

v. Recommended Strategy for Balancing Optimality and Real-Time Performance

Recommended Approach: Alpha-Beta Pruning with Iterative Deepening, Move Ordering, and a Heuristic Evaluation Function

Alpha-Beta Pruning is suitable because it preserves the Minimax result for the searched tree while avoiding the evaluation of branches that cannot affect the final decision.

However, pruning alone does not guarantee that the AI will finish within a strict time limit. It should be combined with additional search-control techniques.

1. Alpha-Beta Pruning

Use Alpha-Beta Pruning to eliminate branches that cannot affect the final minimax decision.

This reduces the number of states evaluated and allows more computation to be directed towards relevant branches.

2. Iterative Deepening

Iterative deepening repeatedly searches the game tree with increasing depth limits.

The AI first searches to depth 1, then depth 2, then depth 3, and so on.

If the time limit is reached, the AI can use the best move found by the last completed search.

This provides a practical way to control computation under real-time constraints.

3. Move Ordering

The AI should examine promising moves first.

For example, it can prioritize actions that appear likely to capture an objective, prevent a major loss, or create a strong tactical advantage.

Good move ordering increases the effectiveness of Alpha-Beta Pruning because it helps the algorithm establish useful alpha and beta bounds earlier.

4. Heuristic Evaluation

When the search reaches its depth limit, the evaluation function estimates the value of the current state.

A carefully designed evaluation function helps the AI distinguish between strategically strong and weak positions without searching to the end of the game.

5. Time-Bounded Search

The AI should have a fixed computation budget for each decision.

It must stop safely before exceeding the deadline and return the best move from the deepest completed search.

The available time can be allocated according to the complexity and importance of the current game state.

6. Transposition Tables

Different sequences of moves may lead to the same game state. A transposition table stores previously calculated state values and search information.

When the same state is encountered again, the AI may reuse the stored information rather than recomputing it.

This can reduce repeated work, provided that the stored information is valid for the current search conditions.

7. Adaptive Search Depth

The AI can use deeper searches when the branching factor is manageable or the decision is strategically important. It can use shallower searches when many actions are available or the deadline is close.

The system should also ensure that its search model is appropriate for the game's mechanics. Standard Minimax is most directly applicable to alternating-turn, adversarial settings. A game with simultaneous actions or continuously changing states may require additional modelling techniques.

Final Recommendation

Alpha-Beta Pruning combined with iterative deepening, move ordering, transposition tables, and a heuristic evaluation function provides a practical balance between decision quality and real-time performance.

This strategy does not guarantee the globally optimal move in the complete game. Instead, it aims to find the best move within the searched tree and available computation budget.

Result

Minimax models competitive decision-making by evaluating possible actions and assuming that the opponent responds strategically. However, its computational cost increases exponentially with search depth and branching factor.

Alpha-Beta Pruning improves efficiency by eliminating branches that cannot affect the final minimax decision. A heuristic evaluation function estimates non-terminal states, while iterative deepening, move ordering, and transposition tables further reduce computation.

Therefore, Alpha-Beta Pruning with iterative deepening and a carefully designed evaluation function is recommended for a game AI operating under strict time limits. It provides strong decision-making within a bounded search while balancing strategic quality and computational efficiency.